Object Oriented Programming

Mihai Dascălu

Strings and Text I/O

Constructing Strings

```
String newString = new String(stringLiteral);

String message = new String("Welcome to Java");
```

Since strings are used frequently, Java provides a shorthand initializer for creating a string:

```
String message = "Welcome to Java";
```

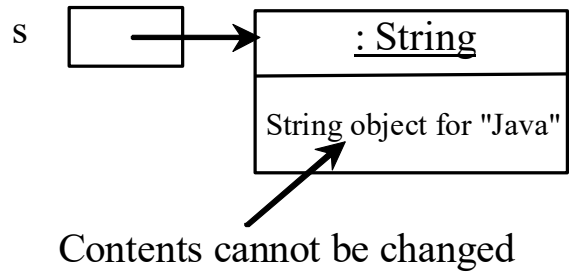
Strings are Immutable

- A String object is immutable => its contents cannot be changed

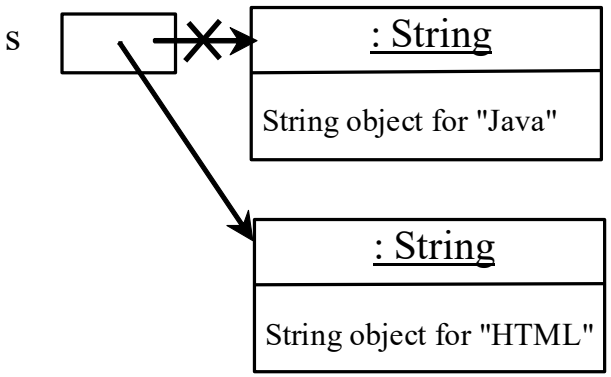
```
String s = "Java";

s = "HTML";
```

After executing `String s = "Java";`



After executing `s = "HTML";`



Interned Strings

Strings are immutable and are frequently used

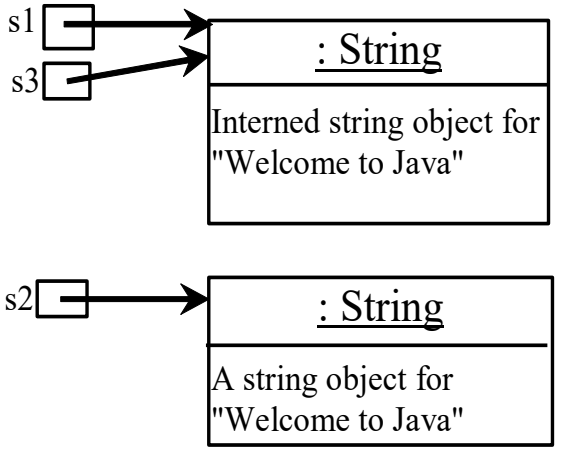
The JVM creates a **unique** instance for string literals with the same character sequence o improve efficiency and save memory => *interned*

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";

System.out.println("s1 == s2 is " + (s1 == s2));
System.out.println("s1 == s3 is " + (s1 == s3));
```



displays

```
s1 == s2 is false

s1 == s3 is true
```

- A new object is created due to the new operator
- If the string initializer is used, no new object is created if the interned object is already created

IntString

String Comparisons (1)

java.lang.String	
+equals(s1: String): boolean	Returns true if this string is equal to string s1.
+equalsIgnoreCase(s1: String): boolean	Returns true if this string is equal to string s1 case-insensitive.
+compareTo(s1: String): int	Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1.
+compareToIgnoreCase(s1: String): int	Same as compareTo except that the comparison is case-insensitive.
+regionMatches(toffset: int, s1: String, offset: int, len: int): boolean	Returns true if the specified subregion of this string exactly matches the specified subregion in string s1.
+regionMatches(ignoreCase: boolean, toffset: int, s1: String, offset: int, len: int): boolean	Same as the preceding method except that you can specify whether the match is case-sensitive.
+startsWith(prefix: String): boolean	Returns true if this string starts with the specified prefix.
+endsWith(suffix: String): boolean	Returns true if this string ends with the specified suffix.

equals()

```
String s1 = new String("Welcome");
String s2 = "Welcome";

if (s1.equals(s2)){
  // s1 and s2 have the same contents
}
if (s1 == s2) {
  // s1 and s2 have the same reference
}
```

compareTo(Object object)

```
String s1 = new String("Welcome");
String s2 = "welcome";
if (s1.compareTo(s2) > 0) {
    // s1 is greater than s2
}
else if (s1.compareTo(s2) == 0) {
   // s1 and s2 have the same contents
}
else
   // s1 is less than s2 (-32)
```

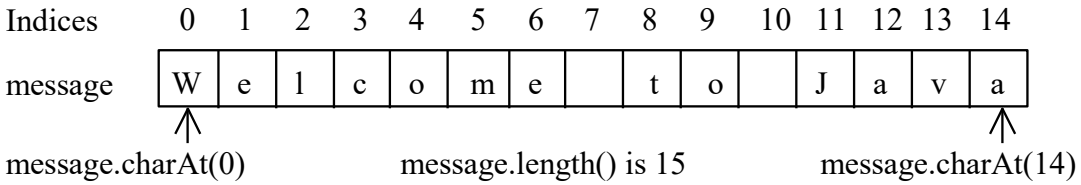
String Length, Characters, and Combining Strings

java.lang.String	
+length(): int	Returns the number of characters in this string.
+charAt(index: int): char	Returns the character at the specified index from this string.
+concat(s1: String): String	Returns a new string that concatenate this string with string s1.

```
message = "Welcome";

message.length() (returns 7)
```

- Do not use `message[0]`
- Use `message.charAt(index)` -> Index starts from `0`




```
String s3 = s1.concat(s2);
String s3 = s1 + s2;
s1 + s2 + s3 + s4 + s5 same as
(((s1.concat(s2)).concat(s3)).concat(s4)).concat(s5);
```

Extracting Substrings

java.lang.String
+subString(beginIndex: int): String +subString(beginIndex: int, endIndex: int): String

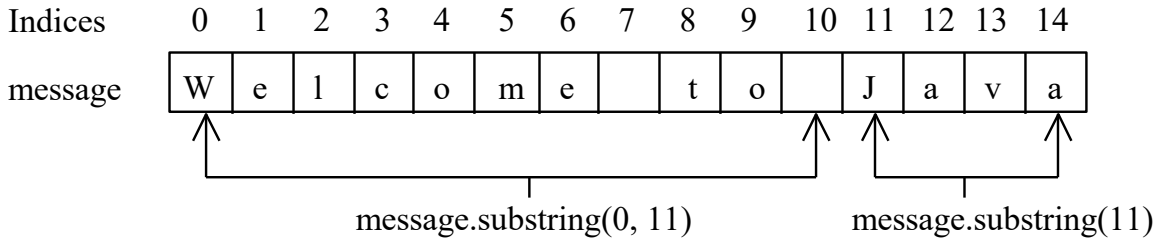
Returns this string's substring that begins with the character at the specified beginIndex and extends to the end of the string.

Returns this string's substring that begins at the specified beginIndex and extends to the character at index endIndex – 1. **Note that the character at endIndex is not part of the substring.**

- Extract a single character from a string using the charAt method
- Extract a substring from a string using the substring method in the String class

```
String s1 = "Welcome to Java";

String s2 = s1.substring(0, 11) + "HTML";
```



Converting, Replacing, and Splitting Strings

java.lang.String	
+toLowerCase(): String	Returns a new string with all characters converted to lowercase.
+toUpperCase(): String	Returns a new string with all characters converted to uppercase.
+trim(): String	Returns a new string with blank characters trimmed on both sides.
+replace(oldChar: char, newChar: char): String	Returns a new string that replaces all matching character in this string with the new character.
+replaceFirst(oldString: String, newString: String): String	Returns a new string that replaces the first matching substring in this string with the new substring.
+replaceAll(oldString: String, newString: String): String	Returns a new string that replace all matching substrings in this string with the new substring.
+split(delimiter: String): String[]	Returns an array of strings consisting of the substrings split by the delimiter.

Examples

"Welcome".toLowerCase() returns a new string, welcome.

"Welcome".toUpperCase() returns a new string, WELCOME.

" Welcome ".trim() returns a new string, Welcome.

"Welcome".replace('e', 'A') returns a new string, WAlcomA.

"Welcome".replaceFirst("e", "AB") returns a new string, WABlcome.

"Welcome".replace("e", "AB") returns a new string, WABlcomAB.

"Welcome".replace("el", "AB") returns a new string, WABcome.

```
String[] tokens = "Java#HTML#Perl".split("#", 0);
for (int i = 0; i < tokens.length; i++)
  System.out.print(tokens[i] + " ");
```

Displays

```
Java HTML Perl
```

Matching, Replacing and Splitting by Patterns

- Match, replace, or split a string by specifying a pattern
  - Extremely useful and powerful feature, commonly known as *regular expression*

```
"Java".matches("Java");

"Java".equals("Java");

"Java is fun".matches("Java.*");

"Java is cool".matches("Java.*");
```

- The replaceAll, replaceFirst, and split methods can be used with a regular expression

```
String s = "a+b$#c".replaceAll("[$+#]", "NNN");

System.out.println(s);
```

- [$+#] specifies a pattern that matches $, +, or #
  => aNNNbNNNNNNc

Regex - Characters

x	The character x
\\	The backslash character
\0n	The character with octal value 0n (0 <= n <= 7)
\0nn	The character with octal value 0nn (0 <= n <= 7)
\0mnn	The character with octal value 0mnn (0 <= m <= 3, 0 <= n <= 7)
\xhh	The character with hexadecimal value 0xhh
\uhhhh	The character with hexadecimal value 0xhhhh
\x{h...h}	The character with hexadecimal value 0xh...h (Character.MIN_CODE_POINT <= 0xh...h <= Character.MAX_CODE_POINT)
\t	The tab character ('\u0009')
\n	The newline (line feed) character ('\u000A')
\r	The carriage-return character ('\u000D')
\f	The form-feed character ('\u000C')
\a	The alert (bell) character ('\u0007')
\e	The escape character ('\u001B')
\cx	The control character corresponding to x

Regex - Character classes

[abc]	a, b, or c (simple class)
[^abc]	Any character except a, b, or c (negation)
[A-Za-z]	a through z or A through Z, inclusive (range)
[a-d[m-p]]	a through d, or m through p: [a-dm-p] (union)
[a-z&&[def]]	d, e, or f (intersection)
[a-z&&[^bc]]	a through z, except for b and c: [ad-z] (subtraction)
[a-z&&[^m-p]]	a through z, and not m through p: [a-lq-z] (subtraction)

Regex - Predefined character classes

.	Any character (may or may not match line terminators)
\d	A digit: [0-9]
\D	A non-digit: [^0-9]
\s	A whitespace character: [\t\n\x0B\f\r]
\S	A non-whitespace character: [^\s]
\w	A word character: [a-zA-Z_0-9]
\W	A non-word character: [^\w]

Regex- Boundary matchers

^	The beginning of a line
$	The end of a line
\b	A word boundary
\B	A non-word boundary
\A	The beginning of the input
\G	The end of the previous match
\Z	The end of the input but for the final terminator, if any
\z	The end of the input

Regex – Greedy quantifiers

X?	X, once or not at all
X*	X, zero or more times
X+	X, one or more times
X{n}	X, exactly n times
X{n,}	X, at least n times
X{n,m}	X, at least n but not more than m times

Regular Expression Syntax

Regular Expression	Matches	Example
x	a specified character x	Java matches Java
.	any single character	Java matches J..a
(ab|cd)	a, b, or c	ten matches t(en|im)
[abc]	a, b, or c	Java matches Ja[uvwx]a
[^abc]	any character except a, b, or c	Java matches Ja[^ars]a
[a-z]	a through z	Java matches [A-M]av[a-d]
[^a-z]	any character except a through z	Java matches Jav[^b-d]
[a-e[m-p]]	a through e or m through p	Java matches [A-G[I-M]]av[a-d]
[a-e&&[c-p]]	intersection of a-e with c-p	Java matches [A-P&&[I-M]]av[a-d]
\d	a digit, same as [1-9]	Java2 matches "Java[\\d]"
\D	a non-digit	$Java matches "[\\D][\\D]ava"
\w	a word character	Java matches "[\\w]ava"
\W	a non-word character	$Java matches "[\\W][\\w]ava"
\s	a whitespace character	"Java 2" matches "Java\\s2"
\S	a non-whitespace char	Java matches "[\\S]ava"
*p**	zero or more occurrences of pattern *p*	Java matches "[\\w]*"
p+	one or more occurrences of pattern *p*	Java matches "[\\w]+"
p?	zero or one occurrence of pattern *p*	Java matches "[\\w]?Java" Java matches "[\\w]?ava"
p{n}	exactly n occurrences of pattern *p*	Java matches "[\\w]{4}"
p{n,}	at least n occurrences of pattern *p*	Java matches "[\\w]{3,}"
p{n,m}	between n and m occurrences (inclusive)	Java matches "[\\w]{1,9}"

Replacing and Splitting Strings

java.lang.String	
+matches(regex: String): boolean	Returns true if this string matches the pattern.
+replaceAll(regex: String, replacement: String): String	Returns a new string that replaces all matching substrings with the replacement.
+replaceFirst(regex: String, replacement: String): String	Returns a new string that replaces the first matching substring with the replacement.
+split(regex: String): String[]	Returns an array of strings consisting of the substrings split by the matches.

```
String s = "Java Java Java ".replaceAll
("v\\w\\W", "wi") ; =>JawiJawiJawi
String s = "Java Java Java".replaceFirst
("v.\\W", "wi") ; =>JawiJava Java
String[] s = "Java1HTML2Perl".split("\\d");
```

Finding a Character or a Substring in a String

java.lang.String	
+indexOf(ch: char): int	Returns the index of the first occurrence of ch in the string. Returns -1 if not matched.
+indexOf(ch: char, fromIndex: int): int	Returns the index of the first occurrence of ch after fromIndex in the string. Returns -1 if not matched.
+indexOf(s: String): int	Returns the index of the first occurrence of string s in this string. Returns -1 if not matched.
+indexOf(s: String, fromIndex: int): int	Returns the index of the first occurrence of string s in this string after fromIndex. Returns -1 if not matched.
+lastIndexOf(ch: int): int	Returns the index of the last occurrence of ch in the string. Returns -1 if not matched.
+lastIndexOf(ch: int, fromIndex: int): int	Returns the index of the last occurrence of ch before fromIndex in this string. Returns -1 if not matched.
+lastIndexOf(s: String): int	Returns the index of the last occurrence of string s. Returns -1 if not matched.
+lastIndexOf(s: String, fromIndex: int): int	Returns the index of the last occurrence of string s before fromIndex. Returns -1 if not matched.

Finding a Character or a Substring in a String

```
"Welcome to Java".indexOf('W') returns 0.

"Welcome to Java".indexOf('x') returns -1.

"Welcome to Java".indexOf('o', 5) returns 9.

"Welcome to Java".indexOf("come") returns 3.

"Welcome to Java".indexOf("Java", 5) returns 11.

"Welcome to Java".indexOf("java", 5) returns -1.

"Welcome to Java".lastIndexOf('a') returns 14.
```

Convert Characters and Numbers to Strings

- The String class provides several static valueOf methods for converting a character, an array of characters, and numeric values to strings
  - Different argument types: char, char[], double, long, int, and float
  - String.valueOf(5.44) => String consisting of characters '5', '.', '4', and '4'

The Character Class

java.lang.Character	
+Character(value: char)	Constructs a character object with char value
+charValue(): char	Returns the char value from this object
+compareTo(anotherCharacter: Character): int	Compares this character with another
+equals(anotherCharacter: Character): boolean	Returns true if this character equals to another
+isDigit(ch: char): boolean	Returns true if the specified character is a digit
+isLetter(ch: char): boolean	Returns true if the specified character is a letter
+isLetterOrDigit(ch: char): boolean	Returns true if the character is a letter or a digit
+isLowerCase(ch: char): boolean	Returns true if the character is a lowercase letter
+isUpperCase(ch: char): boolean	Returns true if the character is an uppercase letter
+toLowerCase(ch: char): char	Returns the lowercase of the specified character
+toUpperCase(ch: char): char	Returns the uppercase of the specified character

Examples

Character charObject = new Character('b');

charObject.compareTo(new Character('a')) returns 1

charObject.compareTo(new Character('b')) returns 0

charObject.compareTo(new Character('c')) returns -1

charObject.compareTo(new Character('d') returns -2

charObject.equals(new Character('b')) returns true

charObject.equals(new Character('d')) returns false

StringBuilder and StringBuffer

- The `StringBuilder`/`StringBuffer` class is an alternative to the `String` class
- In general, a `StringBuilder`/`StringBuffer` can be used wherever a string is used
- `StringBuilder` & `StringBuffer` are more flexible than String
- StringBuffer is practically the same as StringBuilder, except that StringBuilder's methods are not synchronized for thread safety
- Add, insert, or append new contents into a string buffer, whereas the value of a String object is fixed once the String is created

StringBuilder Constructors

java.lang.StringBuilder	
+StringBuilder()	Constructs an empty string builder with capacity 16.
+StringBuilder(capacity: int)	Constructs a string builder with the specified capacity.
+StringBuilder(s: String)	Constructs a string builder with the specified string.

Modifying Strings in the Builder

java.lang.StringBuilder	
+append(data: char[]): StringBuilder	Appends a char array into this string builder.
+append(data: char[], offset: int, len: int): StringBuilder	Appends a subarray in data into this string builder.
+append(v: *aPrimitiveType*): StringBuilder	Appends a primitive type value as a string to this builder.
+append(s: String): StringBuilder	Appends a string to this string builder.
+delete(startIndex: int, endIndex: int): StringBuilder	Deletes characters from startIndex to endIndex.
+deleteCharAt(index: int): StringBuilder	Deletes a character at the specified index.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Inserts a subarray of the data in the array to the builder at the specified index.
+insert(offset: int, data: char[]): StringBuilder	Inserts data into this builder at the position offset.
+insert(offset: int, b: *aPrimitiveType*): StringBuilder	Inserts a value converted to a string into this builder.
+insert(offset: int, s: String): StringBuilder	Inserts a string into this builder at the position offset.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Replaces the characters in this builder from startIndex to endIndex with the specified string.
+reverse(): StringBuilder	Reverses the characters in the builder.
+setCharAt(index: int, ch: char): void	Sets a new character at the specified index in this builder.

Examples

```
stringBuilder = new StringBuilder("Welcome ");
stringBuilder.append("Java");
stringBuilder.insert(11, "HTML and ");
stringBuilder.delete(8, 11) changes the builder to
Welcome Java.
stringBuilder.deleteCharAt(8) changes the builder
to Welcome o Java.
stringBuilder.reverse() changes the builder to
avaJ ot emocleW.
stringBuilder.replace(11, 15, "HTML")
   changes the builder to Welcome to HTML.
stringBuilder.setCharAt(0, 'w') sets the builder
to welcome to Java.
```

The toString, capacity, length, setLength, and charAt Methods

java.lang.StringBuilder	
+toString(): String	Returns a string object from the string builder.
+capacity(): int	Returns the capacity of this string builder.
+charAt(index: int): char	Returns the character at the specified index.
+length(): int	Returns the number of characters in this builder.
+setLength(newLength: int): void	Sets a new length in this builder.
+substring(startIndex: int): String	Returns a substring starting at startIndex.
+substring(startIndex: int, endIndex: int): String	Returns a substring from startIndex to endIndex-1.
+trimToSize(): void	Reduces the storage size used for the string builder.

Main Method Is Just a Regular Method

- You can call a regular method by passing actual parameters
- Can you pass arguments to main? For example, the main method in class B is invoked by a method in A, as shown below:

```
public class A {
  public static void main(String[] args) {
    String[] strings = {"New York",
      "Boston", "Atlanta"};
    B.main(strings);
  }
}
```

```
class B {
  public static void main(String[] args) {
    for (int i = 0; i < args.length; i++)
      System.out.println(args[i]);
  }
}
```

Command-Line Parameters

```
class TestMain {

  public static void main(String[] args) {

  ...

  }

}

java TestMain arg0 arg1 arg2 ... argn
```

- In the main method, get the arguments from `args[0]`, `args[1]`, ..., `args[n]`, which corresponds to `arg0`, `arg1`, ..., `argn` in the command line.

The File Class

- An abstraction to deals with most of the machine-dependent complexities of files and path names in a machine-independent fashion
- The filename is a String
- The File class is a wrapper class for the file name and its directory path

java.io.File	
+File(pathname: String)	Creates a File object for the specified pathname. The pathname may be a directory or a file.
+File(parent: String, child: String)	Creates a File object for the child under the directory parent. child may be a filename or a subdirectory.
+File(parent: File, child: String)	Creates a File object for the child under the directory parent. parent is a File object. In the preceding constructor, the parent is a string.
+exists(): boolean	Returns true if the file or the directory represented by the File object exists.
+canRead(): boolean	Returns true if the file represented by the File object exists and can be read.
+canWrite(): boolean	Returns true if the file represented by the File object exists and can be written.
+isDirectory(): boolean	Returns true if the File object represents a directory.
+isFile(): boolean	Returns true if the File object represents a file.
+isAbsolute(): boolean	Returns true if the File object is created using an absolute path name.
+isHidden(): boolean	Returns true if the file represented in the File object is hidden. The exact definition of *hidden* is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period character '.'.
+getAbsolutePath(): String	Returns the complete absolute file or directory name represented by the File object.
+getCanonicalPath(): String	Returns the same as getAbsolutePath() except that it removes redundant names, such as "." and "..", from the pathname, resolves symbolic links (on Unix platforms), and converts drive letters to standard uppercase (on Win32 platforms).
+getName(): String	Returns the last name of the complete directory and file name represented by the File object. For example, new File("c:\\book\\test.dat").getName() returns test.dat.
+getPath(): String	Returns the complete directory and file name represented by the File object. For example, new File("c:\\book\\test.dat").getPath() returns c:\book\test.dat.
+getParent(): String	Returns the complete parent directory of the current directory or the file represented by the File object. For example, new File("c:\\book\\test.dat").getParent() returns c:\book.
+lastModified(): long	Returns the time that the file was last modified.
+delete(): boolean	Deletes this file. The method returns true if the deletion succeeds.
+renameTo(dest: File): boolean	Renames this file. The method returns true if the operation succeeds.

TestFileClass

Text I/O

- A File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file
- In order to perform I/O, you need to create objects using appropriate Java I/O classes
- The objects contain the methods for reading/writing data from/to a file

Java.IO Mini API

java.io Class	Extends From	Key Constructor(s) Arguments	Key Methods
File	Object	File, String String String, String	createNewFile(), delete(), exists(), isDirectory(), isFile(), list(), mkdir(), renameTo()
FileWriter	Writer	File String	close(), flush(), write()
BufferedWriter	Writer	Writer	close(), flush(), newLine(), write()
PrintWriter	Writer	File (as of Java 5) String (as of Java 5) OutputStream Writer	close(), flush(), format(), printf() print(), println(), write()
FileReader	Reader	File String	read()
BufferedReader	Reader	Reader	read() readLine()
Scanner	Reader	File String	hasNext(), next(), nextInt(), …

Writing Data Using PrintWriter

java.io.PrintWriter	
+PrintWriter(filename: String)	Creates a PrintWriter for the specified file.
+print(s: String): void	Writes a string.
+print(c: char): void	Writes a character.
+print(cArray: char[]): void	Writes an array of character.
+print(i: int): void	Writes an int value.
+print(l: long): void	Writes a long value.
+print(f: float): void	Writes a float value.
+print(d: double): void	Writes a double value.
+print(b: boolean): void	Writes a boolean value.
Also contains the overloaded println methods.	A println method acts like a print method; additionally it prints a line separator. The line separator string is defined by the system. It is \r\n on Windows and \n on Unix.
Also contains the overloaded printf methods.	

WriteData

Reading Data Using Scanner

java.util.Scanner	
+Scanner(source: File)	Creates a Scanner that produces values scanned from the specified file.
+Scanner(source: String)	Creates a Scanner that produces values scanned from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has another token in its input.
+next(): String	Returns next token as a string.
+nextByte(): byte	Returns next token as a byte.
+nextShort(): short	Returns next token as a short.
+nextInt(): int	Returns next token as an int.
+nextLong(): long	Returns next token as a long.
+nextFloat(): float	Returns next token as a float.
+nextDouble(): double	Returns next token as a double.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern.

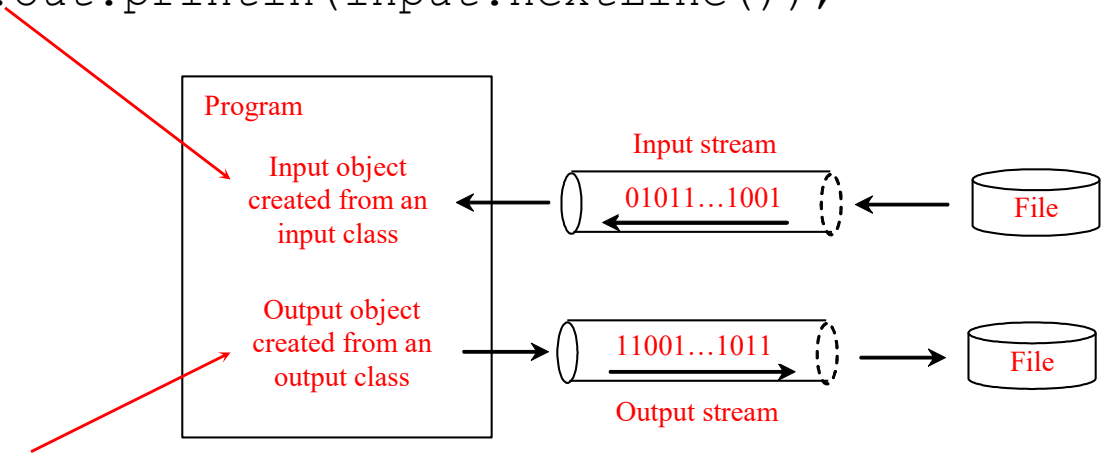
ReadData

Binary I/O

How is I/O Handled in Java?

- A File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file
  - Need to create objects using appropriate Java I/O classes

```
Scanner input = new Scanner(new File("temp.txt"));
System.out.println(input.nextLine());
```



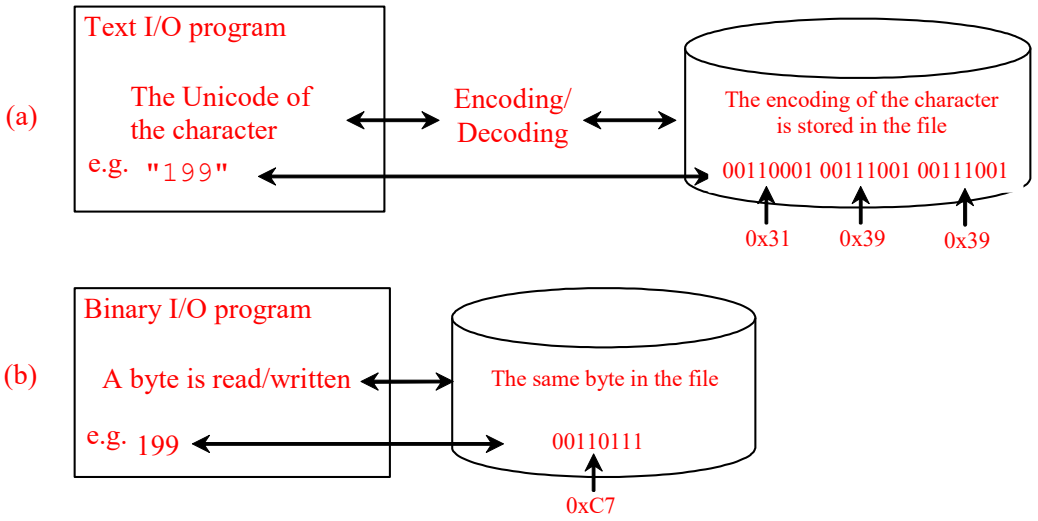

```
PrintWriter output = new PrintWriter("temp.txt");
output.println("Java 101");
output.close();
```

Text File vs Binary File

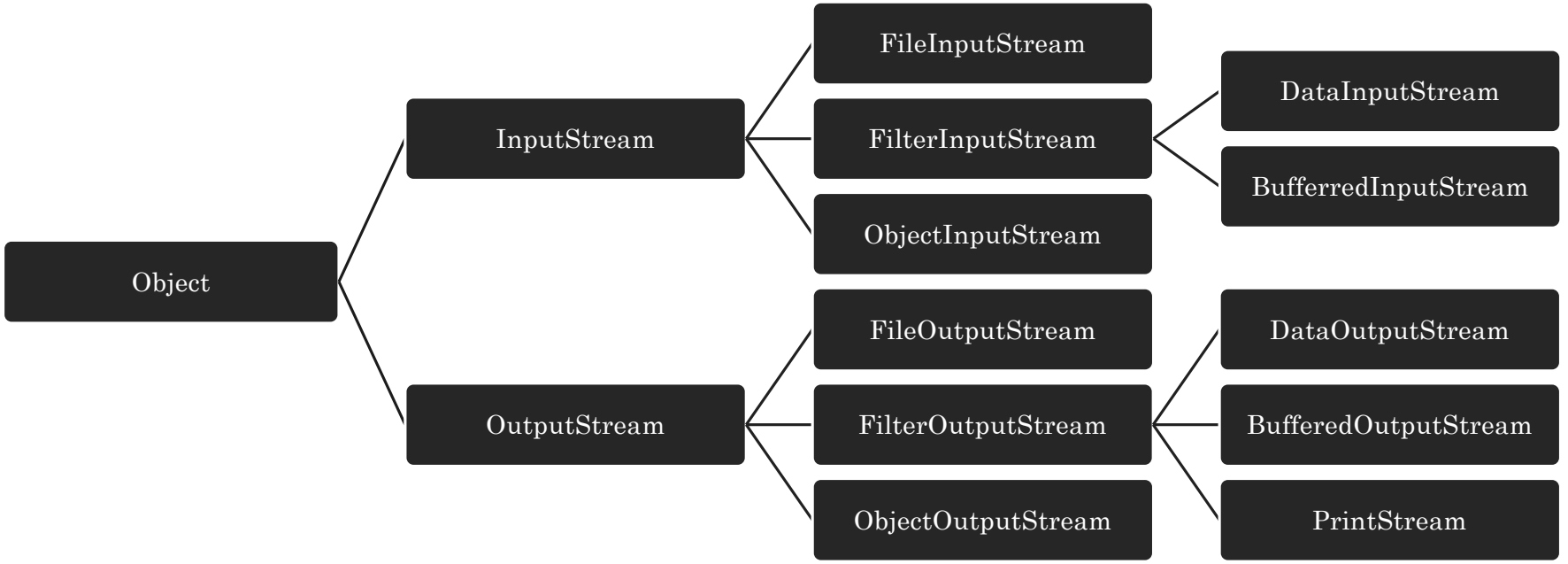
- Data stored in a text file are represented in human-readable form (e.g., .java file)
- Data stored in a binary file are represented in binary form => cannot be read by humans (e.g., .class file)
- Binary files are more efficient to process than text files
  - e.g., int 199 is stored as the sequence of three characters: '1', '9', '9' in a text file vs
    a byte-type value hex C7 in a binary file

Binary I/O

- Text I/O requires encoding and decoding
- The JVM converts a Unicode to a file specific encoding when writing a character and coverts a file specific encoding to a Unicode when reading a character
- Binary I/O does not require conversions
  - When writing a byte to a file, the original byte is copied into the file
  - When reading a byte from a file, the exact byte in the file is returned

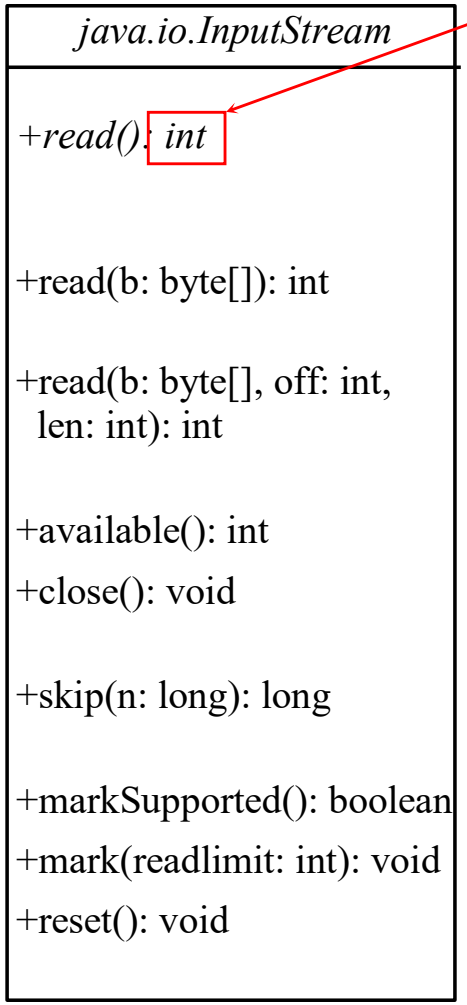


Binary I/O Classes



InputStream

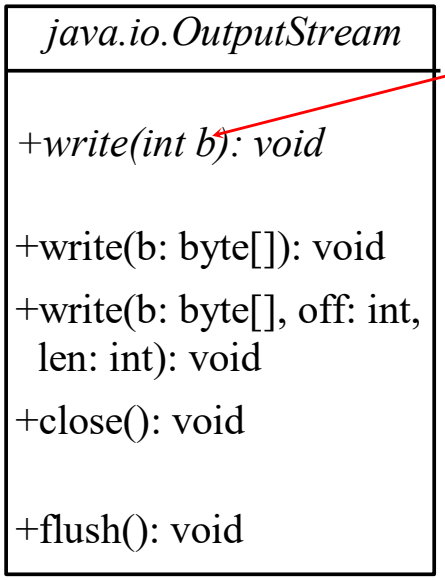
The returned value is a byte as an int type.



+*read()*: *int*	Reads the next byte of data from the input stream. The value byte is returned as an int value in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value –1 is returned.
+read(b: byte[]): int	Reads up to b.length bytes into array b from the input stream and returns the actual number of bytes read. Returns -1 at the end of the stream.
+read(b: byte[], off: int, len: int): int	Reads bytes from the input stream and stores into b[off], b[off+1], …, b[off+len-1]. The actual number of bytes read is returned. Returns -1 at the end of the stream.
+available(): int	Returns the number of bytes that can be read from the input stream.
+close(): void	Closes this input stream and releases any system resources associated with the stream.
+skip(n: long): long	Skips over and discards n bytes of data from this input stream. The actual number of bytes skipped is returned.
+markSupported(): boolean	Tests if this input stream supports the mark and reset methods.
+mark(readlimit: int): void	Marks the current position in this input stream.
+reset(): void	Repositions this stream to the position at the time the mark method was last called on this input stream.

OutputStream

The value is a byte as an int type.



Writes the specified byte to this output stream. The parameter b is an int value. (byte)b is written to the output stream.

Writes all the bytes in array b to the output stream.

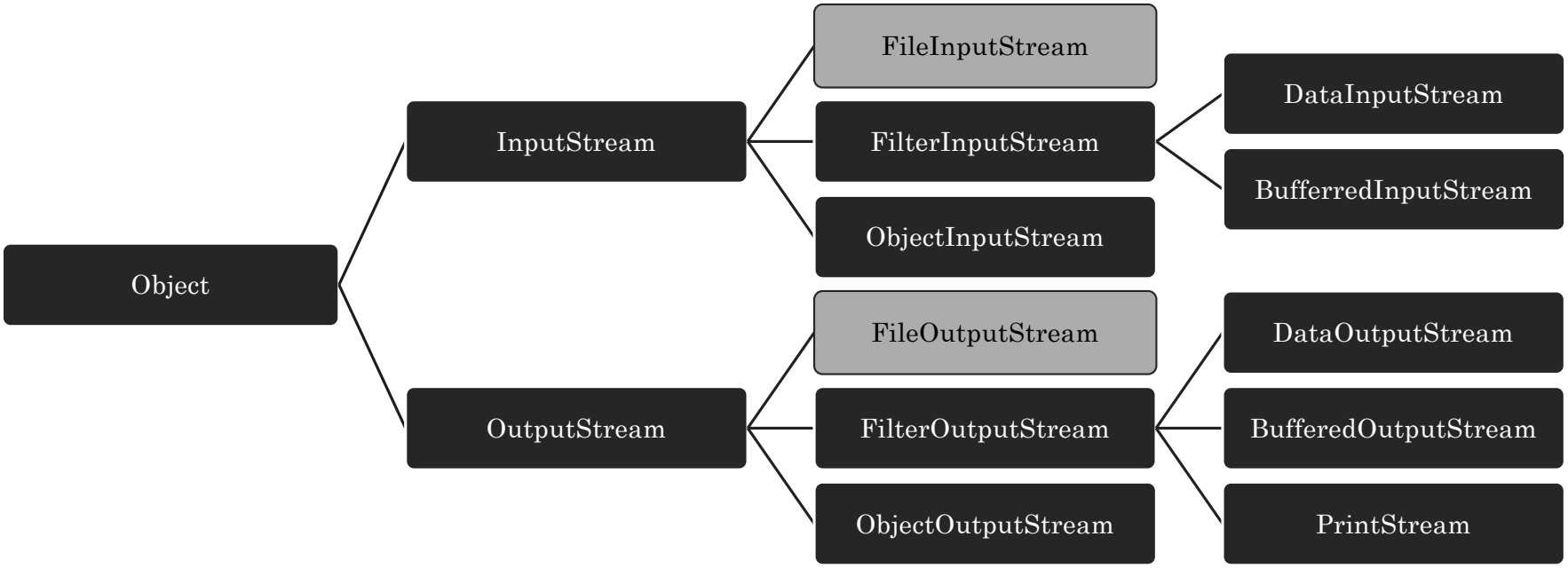
Writes b[off], b[off+1], …, b[off+len-1] into the output stream.

Closes this input stream and releases any system resources associated with the stream.

Flushes this output stream and forces any buffered output bytes to be written out.

FileInputStream/ FileOutputStream

- Associate a binary input/output stream with an external file
- All methods in FileInputStream / FileOutputStream are inherited from its superclasses



FileInputStream

```
public FileInputStream(String name)
public FileInputStream(File file)
```

- A java.io.FileNotFoundException would occur when attempting to create a FileInputStream with a nonexistent file

FileOutputStream

```
public FileOutputStream(String filename)
public FileOutputStream(File file)
public FileOutputStream(String filename, boolean
  append)
public FileOutputStream(File file, boolean append)
```

- If the file does not exist, a new file would be created
- If the file already exists, the first two constructors would delete the current contents in the file
- Use the last two constructors with *append=true* to retain the current content and append new data into the file,

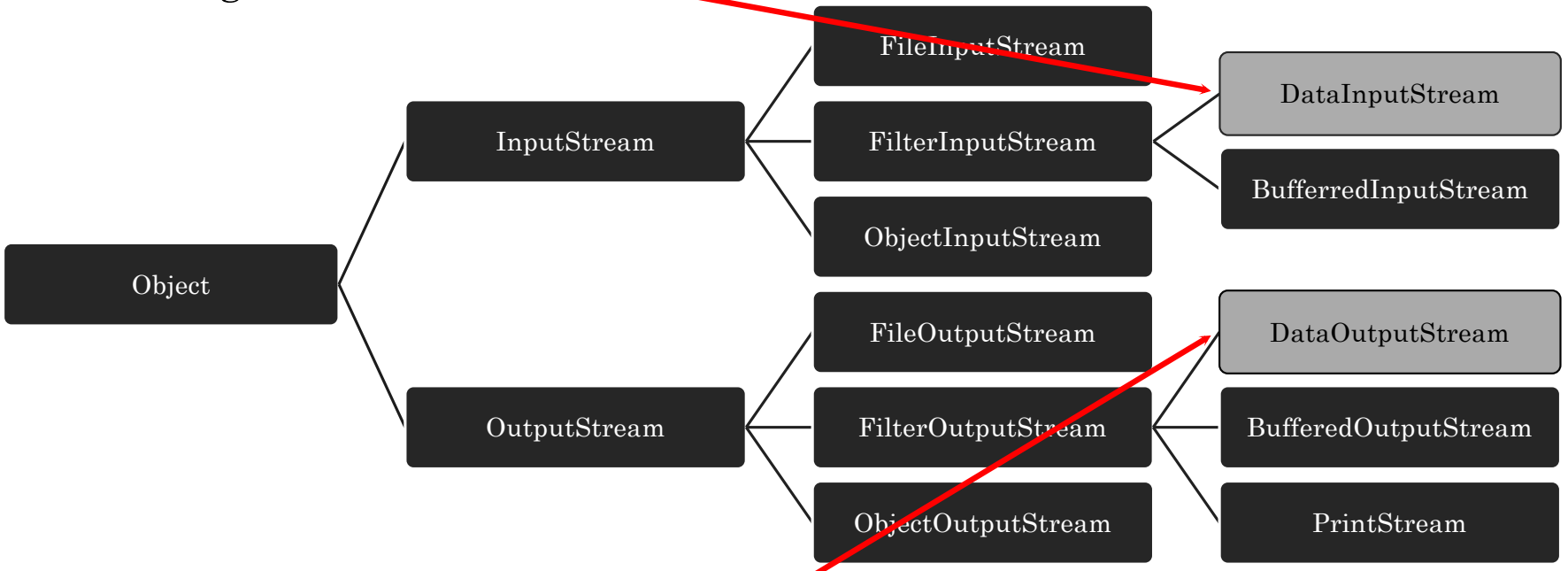
TestFileStream

FilterInputStream/ FilterOutputStream

- Filter bytes for a given purpose
- The basic byte input stream provides a read method that can only be used for reading bytes
- Use a filter class to read integers, doubles, and Strings instead of bytes and characters
- *FilterInputStream* and *FilterOutputStream* are the base classes for filtering data
- Process primitive numeric types: DataInputStream and DataOutputStream to filter bytes

DataInputStream/ DataOutputStream

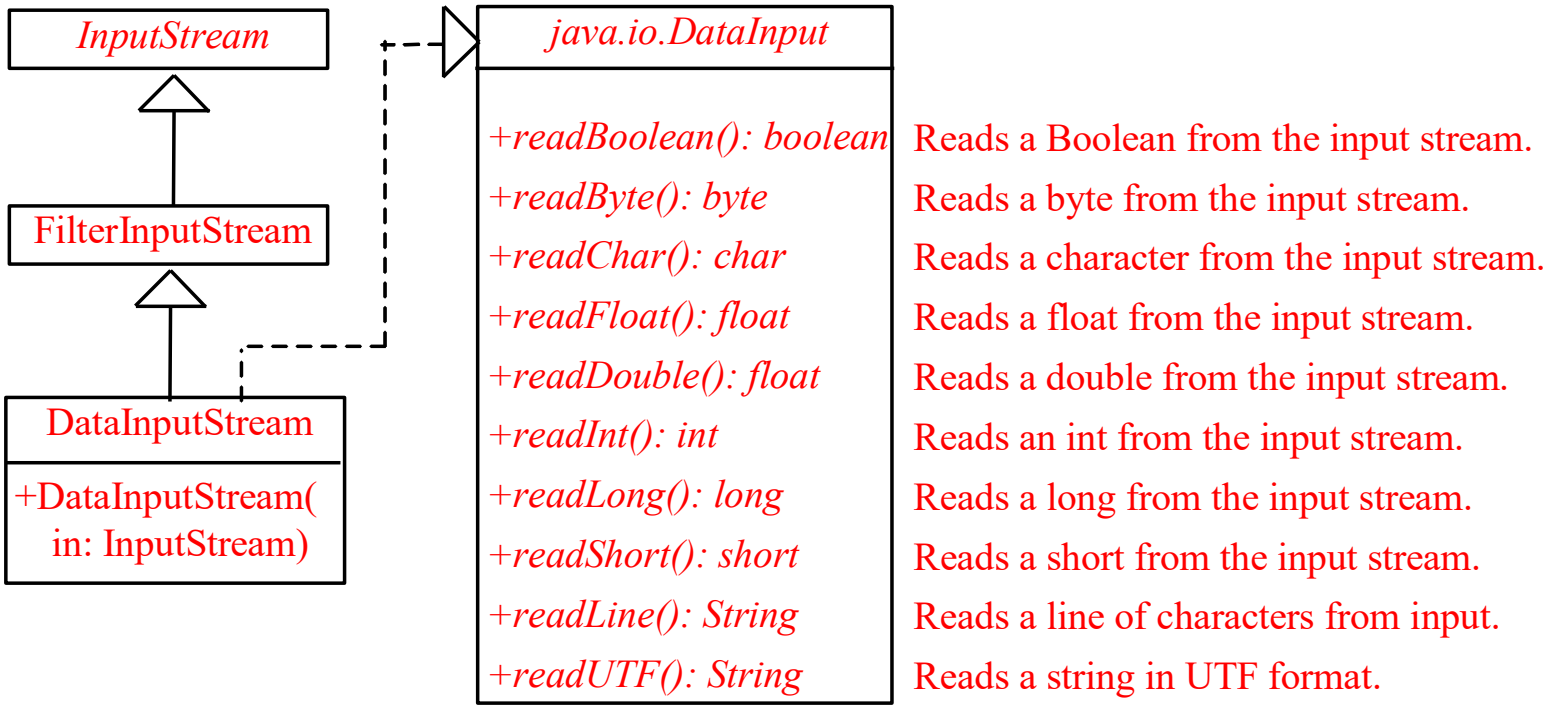
DataInputStream reads bytes from the stream and converts them into appropriate primitive type values or strings



DataOutputStream converts primitive type values or strings into bytes and output the bytes to the stream

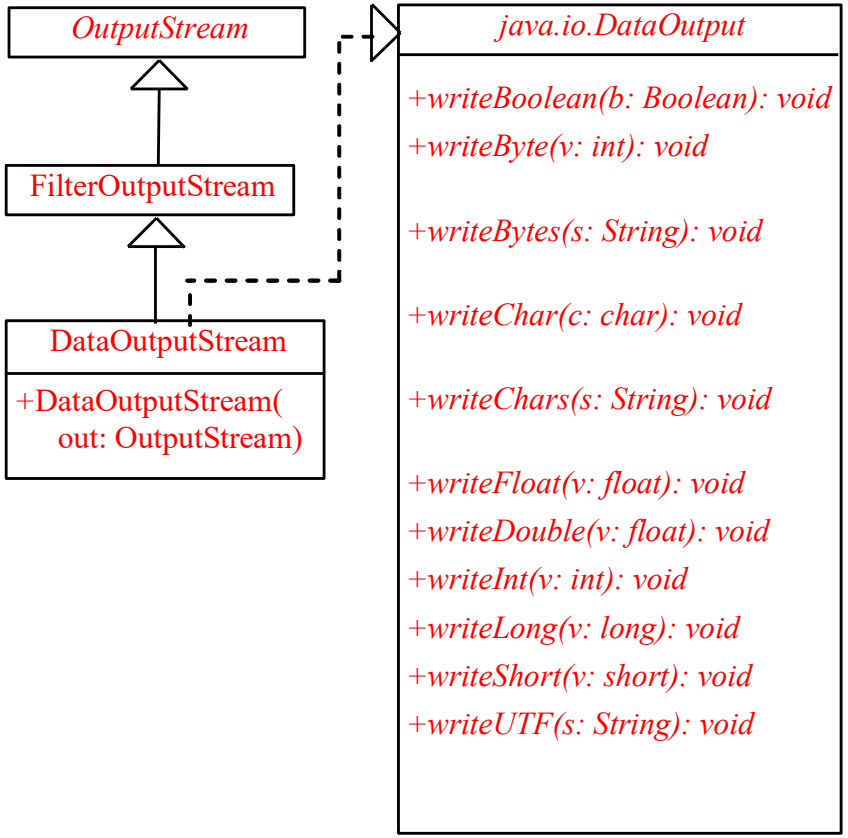
DataInputStream

- DataInputStream extends FilterInputStream and implements the DataInput interface



DataOutputStream

- DataOutputStream extends FilterOutputStream and implements the DataOutput interface



Writes a Boolean to the output stream.

Writes to the output stream the eight low-order bits of the argument v.

Writes the lower byte of the characters in a string to the output stream.

Writes a character (composed of two bytes) to the output stream.

Writes every character in the string s, to the output stream, in order, two bytes per character.

Writes a float value to the output stream.

Writes a double value to the output stream.

Writes an int value to the output stream.

Writes a long value to the output stream.

Writes a short value to the output stream.

Writes two bytes of length information to the output stream, followed by the UTF representation of every character in the string s.

Characters and Strings in Binary I/O

- Writing characters:
  - The writeChar(char *c*) method writes the Unicode character c
  - The writeChars(String *s*) method writes the Unicode for each character in the string *s* to the output
- UTF-8 is a coding scheme that allows systems to operate with both ASCII and Unicode efficiently
- Most operating systems use ASCII (a subset of Unicode)
- Java uses Unicode
- It is a waste to represent an 8-bit ASCII character as a 16-bit Unicode character
- UTF-8 is an alternative scheme that stores a character using 1, 2, or 3 bytes
  - ASCII values (less than 0x7F) are coded in one byte
  - Unicode values less than 0x7FF are coded in two bytes
  - Other Unicode values are coded in three bytes

Using DataInputStream/ DataOutputStream

Data streams are used as wrappers on existing input and output streams to filter data in the original stream:

```
public DataInputStream(InputStream instream)
public DataOutputStream(OutputStream outstream)
DataInputStream infile = new DataInputStream(new
  FileInputStream("in.dat"));
DataOutputStream outfile = new DataOutputStream(new
  FileOutputStream("out.dat"));
```

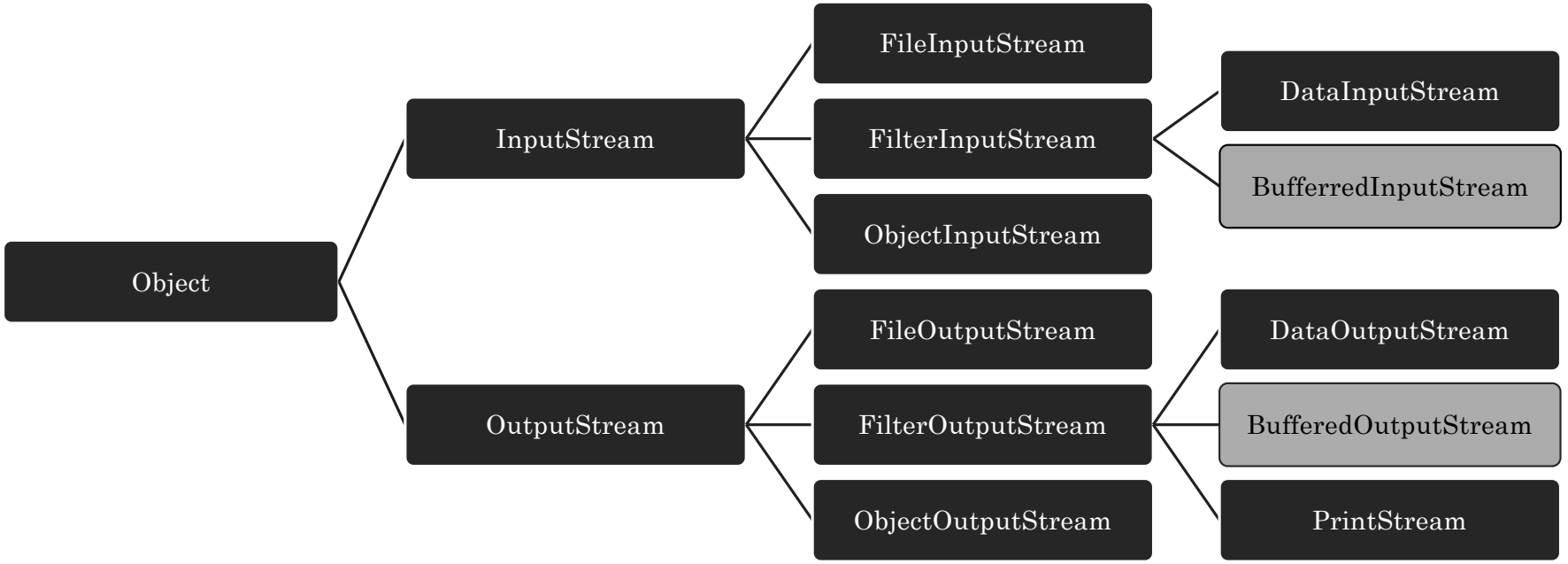
TestDataStream

Checking End of File Order and Format

- TIP: If you keep reading data at the end of a stream, an **EOFException** would occur
- You can use input.available() to check it - input.available() == 0 indicates that it is the end of a file
- CAUTION: You must read the data in the same order and same format in which they are stored
- For example, since names are written in UTF-8 using writeUTF, you must read names using readUTF

BufferedInputStream/ BufferedOutputStream

Use buffers to speed up I/O



BufferedInputStream/BufferedOutputStream do not contain new methods
All the methods BufferedInputStream/BufferedOutputStream are inherited from the InputStream/OutputStream classes.

Example

```
// Create a BufferedInputStream
public BufferedInputStream(InputStream in)
public BufferedInputStream(InputStream in, int
  bufferSize)

// Create a BufferedOutputStream
public BufferedOutputStream(OutputStream out)
public BufferedOutputStream(OutputStreamr out,
  int bufferSize)
```

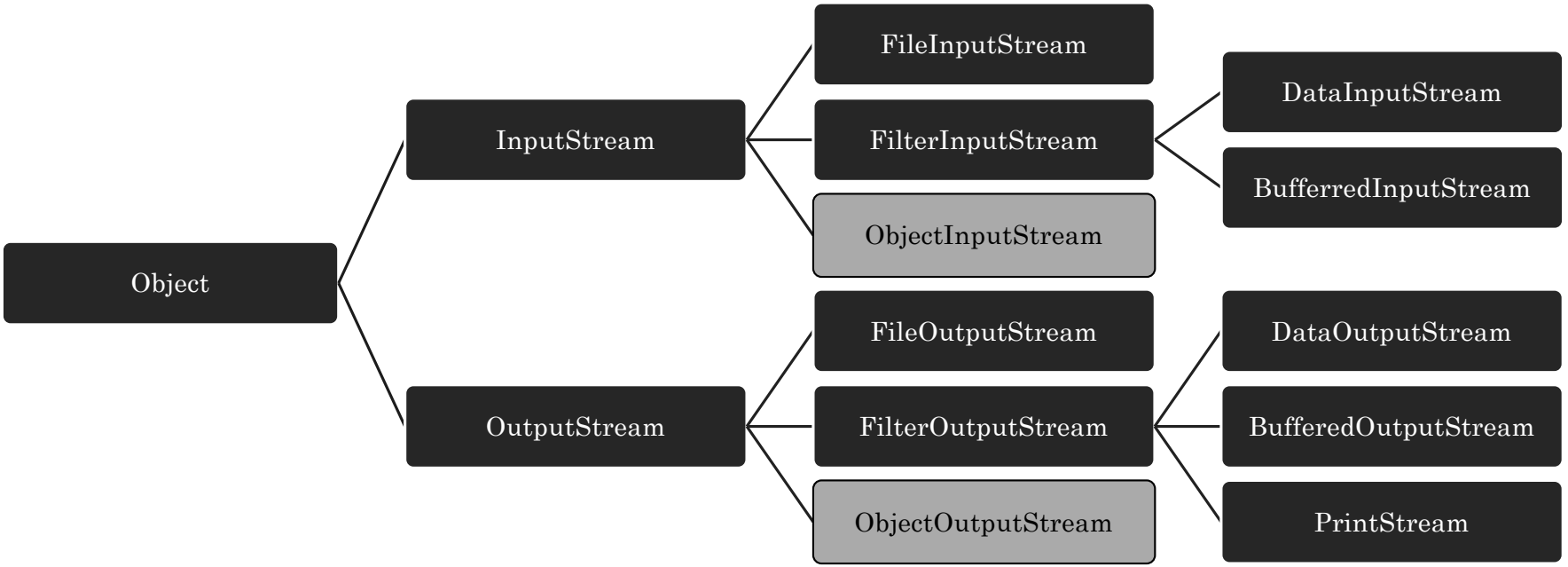
Copy

PrintStream

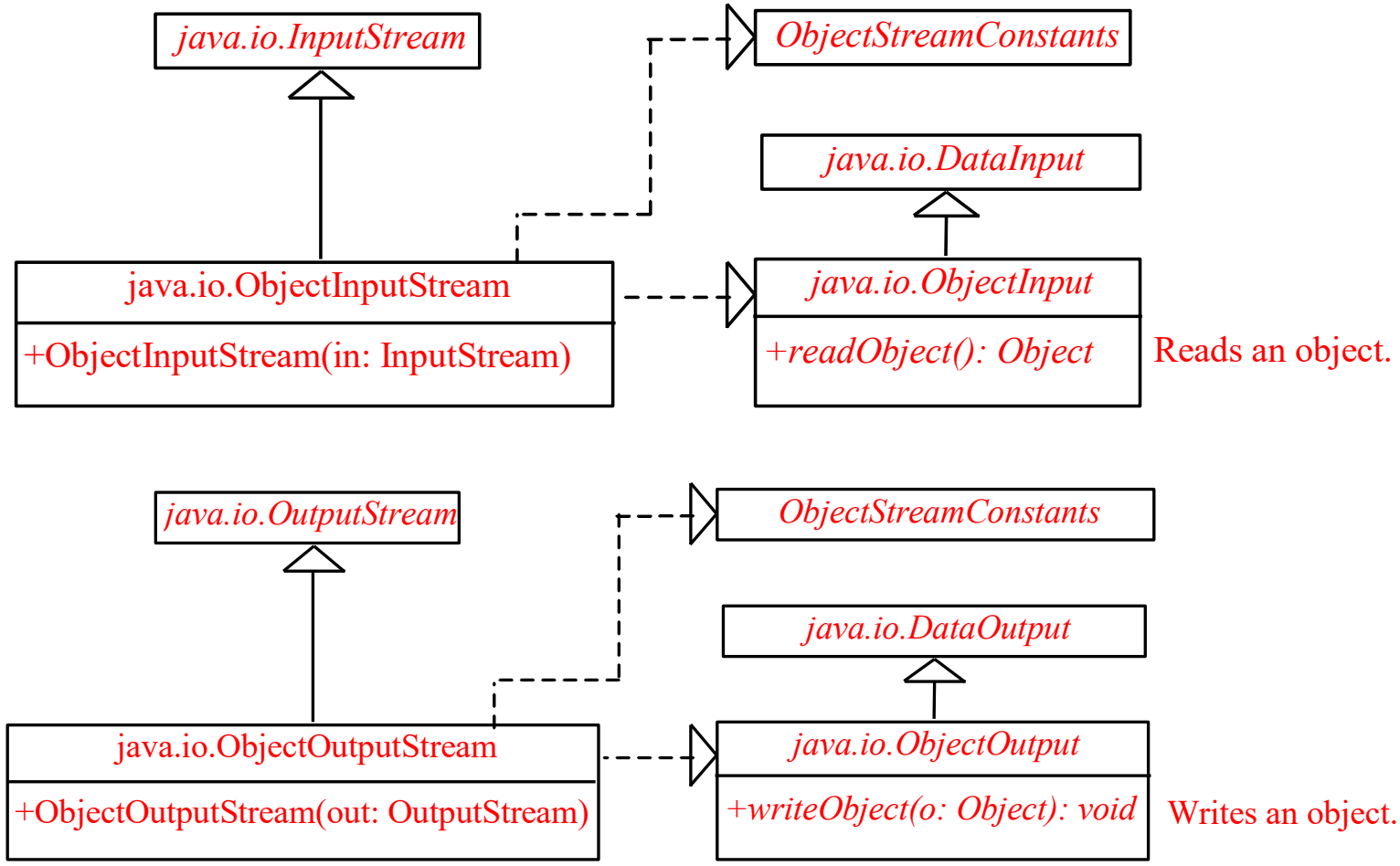
- Adds functionality to another output stream:
  - Print representations of various data values conveniently
- Never throws an IOException
  - Instead, exceptional situations merely set an internal flag that can be tested via the checkError method
- Optionally, a PrintStream can be created so as to flush automatically;
  - i.e., the flush method is automatically invoked:
    - After a byte array is written
    - One of the println methods is invoked, or
    - A newline character or byte ('\n') is written
- Real use case: System.out, .err

Object I/O

- Data I/O Stream enables I/O for **primitive type** values and strings
- Object I/O Stream enables I/O for **objects**, in addition for primitive type values and strings



Object I/O Stream



Using Object Streams

- Wrap an Object I/O Stream on any I/O Stream using the following constructors:

```
// Create an ObjectInputStream
public ObjectInputStream(InputStream in)

// Create an ObjectOutputStream
public ObjectOutputStream(OutputStream out)
```

TestObjectStream

The `Serializable` Interface

- Not all objects can be written to an output stream
- *Serializable* = objects that can be written to an object stream is said to be
  - A serializable object is an instance of the java.io.Serializable interface
- The Serializable interface is a **marker** interface - it has no methods
  - No need to add additional code that implements Serializable
- Implementing this interface enables the Java serialization mechanism to automate the process of storing the objects and arrays

The transient Keyword (1)

- If an object contains non-serializable instance data fields, it cannot be serialized
- Use the **transient** keyword to mark these data fields and tell the JVM to ignore these fields

The `transient` Keyword (2)

```
public class Foo implements java.io.Serializable {

  private int v1;

  private static double v2;

  private transient A v3 = new A();

}

class A { } // A is not serializable
```

- When an object of the Foo class is serialized, only variable v1 is serialized
- Variable v2 is not serialized because it is a static variable, and variable v3 is not serialized because it is marked transient
- If v3 were not marked transient, a java.io.NotSerializableException would occur

SerializedDog

Serializing Arrays

- An array is serializable if all its elements are serializable
- Thus, an entire array can be saved using writeObject into a file and later restored using readObject

TestObjectStreamForArray

Trips & Tricks

- It is okay for a class to implement Serializable, even if its superclass does not
- However, when deserializing such an object, the non-serializable superclass must run its constructor
- Constructors do not run on deserialized classes that implement Serializable

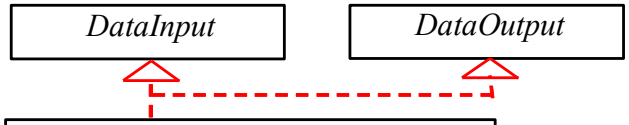
SerializedCat

Random Access Files

Random Access Files

- All streams used so far are known as read-only or write-only streams
- The external files of these streams are sequential files that cannot be updated without creating a new file
- It is often necessary to modify files or to insert new records into files
- Java provides the RandomAccessFile class to allow a file to be read from and write to at random locations

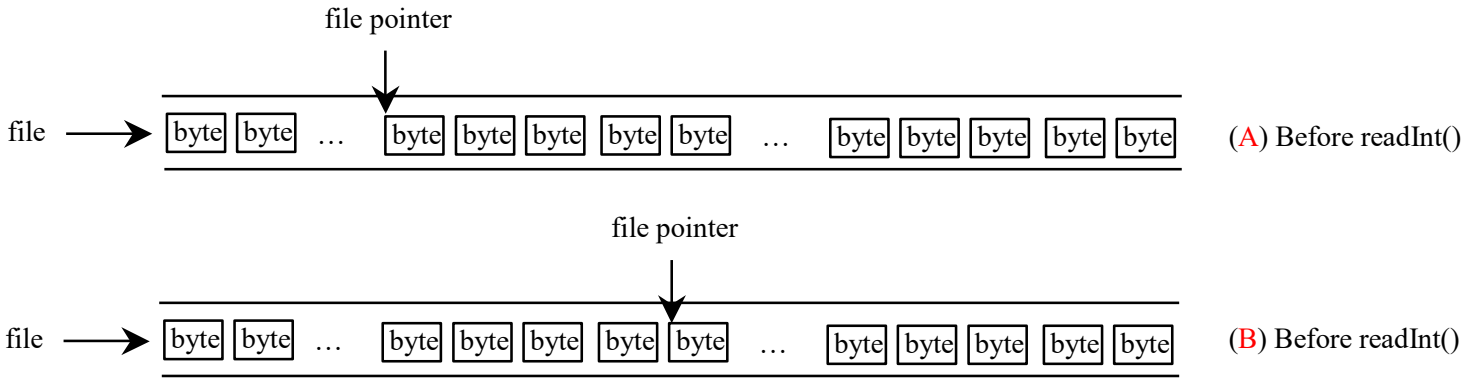
RandomAccessFile



java.io.RandomAccessFile	
+RandomAccessFile(file: File, mode: String)	Creates a RandomAccessFile stream with the specified File object and mode.
+RandomAccessFile(name: String, mode: String)	Creates a RandomAccessFile stream with the specified file name string and mode.
+close(): void	Closes the stream and releases the resource associated with the stream.
+getFilePointer(): long	Returns the offset, in bytes, from the beginning of the file to where the next read or write occurs.
+length(): long	Returns the length of this file.
+read(): int	Reads a byte of data from this file and returns –1 an the end of stream.
+read(b: byte[]): int	Reads up to b.length bytes of data from this file into an array of bytes.
+read(b: byte[], off: int, len: int) : int	Reads up to len bytes of data from this file into an array of bytes.
+seek(long pos): void	Sets the offset (in bytes specified in pos) from the beginning of the stream to where the next read or write occurs.
+setLength(newLength: long): void	Sets a new length of this file.
+skipBytes(int n): int	Skips over n bytes of input discarding the skipped bytes.
+write(b: byte[]): void	Writes b.length bytes from the specified byte array to this file, starting at the current file pointer.
+write(byte b[], int off, int len)	
+write(b: byte[], off: int, len: int): void	Writes len bytes from the specified byte array starting at offset off to this file.

File Pointer

- A random access file consists of a sequence of bytes
- The file pointer is positioned & a read or write operation takes place at the location of the file pointer
- When a file is opened, the file pointer sets at the beginning of the file
- When you read or write data to the file, the file pointer moves forward to the next data



RandomAccessFile Methods (1)

- Many methods in RandomAccessFile are the same as those in DataInputStream and DataOutputStream
  - readInt()
  - readLong()
  - writeDouble()
  - readLine()
  - writeInt()
  - writeLong()

RandomAccessFile Specific Methods

- void seek(long pos) throws IOException;
  - Sets the offset from the beginning of the RandomAccessFile stream to where the next read or write occurs
- long getFilePointer() throws IOException;
  - Returns the current offset, in bytes, from the beginning of the file to where the next read or write occurs
- long length() throws IOException
  - Returns the length of the file
- final void writeChar(int v) throws IOException
  - Writes a character to the file as a two-byte Unicode, with the high byte written first
- final void writeChars(String s) throws IOException
  - Writes a string to the file as a sequence of characters

RandomAccessFile Constructor

- ```
  RandomAccessFile raf =
  new RandomAccessFile("test.dat", "rw");
  ```

```
//allows read and write
```

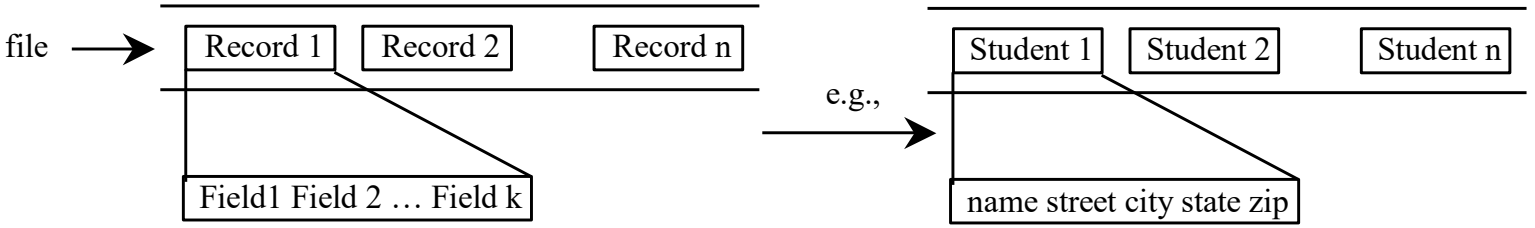
- ```
  RandomAccessFile raf =
  new RandomAccessFile("test.dat", "r");
  ```

```
//read only
```

TestRandomAccessFile

Fixed Length String I/O (1)

- Random access files are often used to process files of records
- For convenience, fixed-length records are used in random access files so that a record can be located easily
- A record consists of a fixed number of fields
  - A field can be a string or a primitive data type
  - A string in a fixed-length record has a maximum size
  - If a string is smaller than the maximum size, the rest of the string is padded with blanks.



Fixed Length String I/O (2)

```
/** Read fixed number of characters from a DataInput stream */
public static String readFixedLengthString(int size,
    DataInput in) throws IOException {
  // Declare an array of characters
  char[] chars = new char[size];
  // Read fixed number of characters to the array
  for (int i = 0; i < size; i++)
    chars[i] = in.readChar();
  return new String(chars);
}

/** Write fixed number of characters to a DataOutput stream */
public static void writeFixedLengthString(String s, int size,
    DataOutput out) throws IOException {
  char[] chars = new char[size];
  // Fill in string with characters
  s.getChars(0, Math.min(s.length(), size), chars, 0);
  // Fill in blank characters in the rest of the array
  for (int i = Math.min(s.length(), size); i < chars.length; i++)
    chars[i] = ' ';
  // Create and write a new string padded with blank characters
  out.writeChars(new String(chars));
}
```

try-with-resources Statement

- A try statement that declares one or more resources
- Ensures that each resource is closed at the end of the statement
- Any object that implements java.lang.AutoCloseable, which includes all java.io.Closeable, can be used as a resource

```
try (BufferedReader br = new BufferedReader(new FileReader(path))) {
  return br.readLine();
}
```

```
BufferedReader br = new BufferedReader(new FileReader(path));
try {
 return br.readLine();
} finally {
 br.close();
}
```

Recap

String, StringBuffer, and StringBuilder – Summary (1)

- String objects are immutable, and String reference variables are not
- If you create a new String without assigning it, that value will be lost
- If you redirect a String reference to a new String, the old String can be lost
- String methods use zero-based indexes, except for the second argument of substring()
- Chained methods are evaluated from left to right

String, StringBuffer, and StringBuilder – Summary (2)

- The String class is final – its methods can't be overridden
- When the JVM finds a String literal, it is added to the String literal pool
- Strings have a method: length();
  - Arrays have an attribute named length
- The StringBuffer's API is the same as the new StringBuilder's API, except that StringBuilder's methods **are not synchronized** for thread safety
- StringBuilder methods **should run faster** than StringBuffer methods

String, StringBuffer, and StringBuilder – Summary (3)

- StringBuffer & StringBuilder:
  - They are mutable—they can change without creating a new object
  - StringBuffer methods act on the invoking object and objects can change without an explicit assignment StringBuffer equals() is not overridden; it doesn't compare values
- String: charAt(), concat(), equalsIgnoreCase(), length(), replace(), substring(), toLowerCase(), toString(), toUpperCase(), and trim()
- StringBuffer: append(), delete(), insert(), reverse(), and toString()

File I/O – Summary (1)

- A new File object does not mean there is a new file on the hard drive
- File objects can represent either a file or a directory
- The File class manages files and directories (add, rename, and delete)
- The methods createNewFile() and mkdir() add entries to your file system
- FileWriter and FileReader are low-level I/O classes
  - Use them to write and read files, but they should usually be wrapped

File I/O – Summary (2)

- Classes in java.io are designed to be "chained" or "wrapped"
- It is very common to "wrap" a BufferedReader around a FileReader or a BufferedWriter around a FileWriter to get access to higher-level methods
- PrintWriters can be used to wrap other Writers, but as of Java 5 they can be built directly from Files or Strings
- Java 5+ PrintWriters have new append(), format(), and printf() methods

Serialization – Summary (1)

- A class must implement Serializable before its objects can be serialized
- The ObjectOutputStream.writeObject() serializes objects, whereas the ObjectInputStream.readObject() deserializes objects
- If an instance variable is marked as ***transient***, it will not be serialized even thought the rest of the object's state will be
- You can supplement a class's automatic serialization process by implementing the writeObject() and readObject() methods

Serialization – Summary (2)

- If overwriting read/write Object, embedding calls to defaultWriteObject() and defaultReadObject(), will handle the part of serialization that happens normally
- If a superclass implements Serializable, then its subclasses do automatically
- If a superclass does not implement Serializable, then the *superclass constructor will be invoked*, along with its superconstructor(s), when a subclass object is deserialized

But what is New? NIO.2

- New I/O APIs
- A subset of the new file system API, alongside Path APIs (Java 7+)
- Central abstractions :
  - **Buffers** - containers for data
  - **Charsets** and their associated decoders and encoders, which translate between bytes and Unicode characters
  - **Channels** of various types, which represent connections to entities capable of performing I/O operations
  - **Selectors** and **selection keys**, which together with selectable channels define a multiplexed, non-blocking I/O

Why NIO?

- Handle symbolic links
- Asynchronous I/O
- Limited functionalities of File (e.g., no copy, resolve, relativize, normalize)

```
InputStream in = new BufferedInputStream(new
FileInputStream(original));
OutputStream out = new BufferedOutputStream(new
FileOutputStream(copied))) {
byte[] buffer = new byte[1024];
int lengthRead;
while ((lengthRead = in.read(buffer)) > 0) {
   out.write(buffer, 0, lengthRead);
   out.flush();
}
```

```
Path copied = Paths.get("src/test/resources/copiedWithNio.txt");
Path originalPath = original.toPath();
Files.copy(originalPath, copied,
StandardCopyOption.REPLACE_EXISTING);
```